

TEKNİK RAPOR

Trino Secure

Kurulum, Konfigürasyon ve Yetkilendirme Rehberi

Tek Instance ve Cluster Kurulumları için Kapsamlı Referans

Konu:

Trino Dağıtık Sorgulama Motorunda Güvenlik ve Konfigürasyon Yönetimi

Kapsam:

Kimlik Doğrulama · Yetkilendirme · Kaynak Yönetimi · Katalog Konfigürasyonu · API

Trino Sürümü:

481 (Docker Image: trinodb/trino:481)

İçindekiler

1. Yönetici Özeti.....	3
2. Trino Güvenlik Mimarisi.....	4
3. Kurulum Ön Hazırlığı ve Dizin Yapısı.....	6
4. Konfigürasyon Dosyaları — Detaylı Referans.....	7
4.1. node.properties.....	7
4.2. jvm.config.....	8
4.3. config.properties.....	9
4.4. log.properties.....	12
4.5. access-control.properties ve rules.json.....	13
4.6. password-authenticator.properties ve password.db.....	17
4.7. group-provider.properties ve group.txt.....	18
4.8. resource-groups.properties ve resource-groups.json.....	19
4.9. Katalog Konfigürasyonları.....	22
4.10. TLS Sertifikaları ve Keystore Yönetimi.....	25
5. Trino API Endpoint Referansı.....	26
6. Web UI ve Yönetim Arayüzü.....	28
7. Tek Instance ve Cluster Kurulumları — Karşılaştırma.....	29
8. Kurulum Adımları ve Sıralı Uygulama.....	31
9. Doğrulama, Test ve İzleme.....	33
10. Restart Gereksinimleri ve Değişiklik Yönetimi.....	34
11. Sonuç ve Öneriler.....	35

1. Yönetici Özeti

Trino, birden fazla veri kaynağına aynı SQL diliyle sorgu atmayı sağlayan açık kaynak, dağıtık bir sorgu motorudur. Varsayılan kurulumunda güvenlik katmanı **aktif değildir**: HTTP üzerinden çalışır, kimlik doğrulama yapılmaz ve X-Trino-User header'ında iletilen değer sorgulayan kullanıcı olarak kabul edilir. Bu, geliştirme ortamı için pratik ancak üretim ortamı için kabul edilemez bir durumdur.

Bu rapor, Trino kurulumunun "secure" olarak adlandırılan modda çalıştırılması için gereken tüm konfigürasyon dosyalarının işlevini, syntax'ını ve kullanımını detaylı olarak açıklamaktadır. Rapor ayrıca **tek instance** (development, PoC) ile **çoklu-node cluster** (üretim) kurulumları arasındaki mimari ve konfigürasyon farklarını karşılaştırmaktadır.

Rapor Kapsamı

- Trino güvenlik mimarisinin üç katmanı: encryption, authentication, authorization
- Ana konfigürasyon dosyalarının (node, jvm, config, log) detaylı işlev ve syntax açıklaması
- Dosya-tabanlı erişim kontrolü (rules.json) ve column/row-level security
- Password authentication ve group provider yapılandırması
- Resource groups ile kaynak kotası yönetimi
- Katalog konfigürasyonları: PostgreSQL, Hive, Elasticsearch, MinIO, Kafka
- TLS/HTTPS sertifika üretimi ve yönetimi
- REST API endpoint referansı ve programatik erişim
- Tek instance ve cluster kurulumlarının karşılaştırmalı analizi
- Doğrulama, test ve izleme prosedürleri

Hedef Kitle

Bu rapor, Trino ile üretim veri altyapıları kuran veri mühendisleri, DevOps ve SRE ekipleri, güvenlik ve yetkilendirme politikalarını yöneten sistem yöneticileri için hazırlanmıştır. Okuyucunun temel Linux sistem yönetimi, JVM tabanlı servislerin işleyişi ve SQL bilgisine sahip olduğu varsayılmaktadır.

2. Trino Güvenlik Mimarisi

Trino güvenliği üç bağımsız katmandan oluşur. Her katman ayrı ayrı yapılandırılır ve devre dışı bırakılabilir. Kurulumun "secure" sayılması için her üç katmanın da uygun şekilde aktifleştirilmesi gerekir.

2.1. Encryption Katmanı (Şifreleme)

Client ile Trino coordinator arasındaki ve — opsiyonel olarak — coordinator ile worker'lar arasındaki tüm iletişimin **TLS (HTTPS)** ile şifrlenmesini kapsar. Password authentication kullanılacaksa **HTTPS zorunludur**; Trino, HTTP üzerinden parola bazlı kimlik doğrulama isteğini reddeder.

- Client → Coordinator: kullanıcı arayüzü, CLI, JDBC/ODBC, HTTP API çağrıları
- Coordinator ↔ Worker: internal communication (task submission, exchange, discovery)
- Coordinator/Worker → Data Source: her connector kendi TLS/SSL ayarlarına sahiptir (PostgreSQL SSL, Elasticsearch TLS, HDFS RPC encryption vb.)

2.2. Authentication Katmanı (Kimlik Doğrulama)

Sorgu gönderen kullanıcının kimliğinin doğrulanmasıdır. Trino aşağıdaki authentication tiplerini destekler:

Tip	Açıklama	Kullanım
PASSWORD	Kullanıcı adı + parola. Dosya (password.db) veya LDAP'tan doğrulanır.	Dahili kullanıcılar, dosya bazlı yönetim
CERTIFICATE	İstemci X.509 sertifikası ile mTLS.	Servis-to-servis, otomasyon
OAuth2	OAuth2/OIDC provider (Keycloak, Okta, Azure AD, Google).	SSO, kurumsal kimlik yönetimi
JWT	İmzalı JWT token doğrulaması.	API-first, mikroservis mimarileri
KERBEROS	Kerberos ticket bazlı.	Hadoop ekosistemi, kurumsal AD
HEADER	Reverse proxy tarafından set edilen HTTP header.	Proxy arkasında merkezi auth

Birden fazla authentication tipi virgülle listelenerek aynı anda aktif edilebilir. Sistem sırayla dener, ilk başarılı olan geçerli sayılır. Örneğin `http-server.authentication.type=OAuth2, PASSWORD` konfigürasyonu, önce OAuth2 dener, başarısız olursa password authentication ile devam eder.

2.3. Authorization ve Access Control Katmanı

Kimlik doğrulaması yapılan kullanıcının hangi katalog/şema/tablo/sütuna hangi izinlerle erişebileceğini belirler. Trino iki seviyede access control sunar:

Sistem Seviyesi (Coarse-Grained)

- `allow-all` — Herkes her şeye erişir (varsayılan, üretim için uygun değil)
- `read-only` — Sadece SELECT ve SHOW komutları
- `file` — JSON dosyadan (rules.json) tanımlı kurallar
- `opa` — Open Policy Agent üzerinden Rego policy'leri

- ranger — Apache Ranger entegrasyonu (audit destekli)

Katalog Seviyesi (Fine-Grained)

Her katalog için ayrı bir kural dosyası tanımlanabilir. Bu, connector-spesifik yetkilendirme sağlar ve modüler yapılarda tercih edilir. Katalog dosyasında `<connector>.security=file` satırı ile aktifleştirilir.

2.4. Güvenlik Katmanlarının İlişkisi

Bir sorgunun Trino cluster'ında gezdiği yol, güvenlik katmanlarının nasıl birlikte çalıştığını gösterir:

1. Client, coordinator'a HTTPS üzerinden bağlanır (encryption).
2. Coordinator, credentials'i doğrular (authentication).
3. Access control eklentisi kullanıcının kaynaklara erişimini kontrol eder (authorization).
4. Group provider, kullanıcının hangi gruplarda olduğunu belirler.
5. Resource group manager, sorguyu uygun kuyruğa atar (kota kontrolü).
6. Query planner, execution plan üretir ve worker'lara dağıtır (internal-communication.shared-secret ile).
7. Worker'lar connector aracılığıyla veri kaynağına gider (connector-level auth).
8. Sonuçlar coordinator'da toplanır ve client'a döndürülür.

3. Kurulum Ön Hazırlığı ve Dizin Yapısı

Trino kurulumu için standart dizin yapısı aşağıdaki gibidir. Docker container kullanılması durumunda bu dizin host makinede oluşturulur ve container'a bind mount olarak bağlanır.

3.1. Standart Dizin Yapısı

```
/opt/trino/
├── etc/
│   ├── node.properties
│   ├── jvm.config
│   ├── config.properties
│   ├── log.properties
│   ├── access-control.properties
│   ├── password-authenticator.properties
│   ├── group-provider.properties
│   ├── resource-groups.properties
│   ├── password.db
│   ├── group.txt
│   ├── rules.json
│   ├── resource-groups.json
│   └── catalog/
│       ├── postgres.properties
│       ├── hive.properties
│       ├── elasticsearch.properties
│       ├── mysql.properties
│       └── memory.properties
├── tls/
│   ├── keystore.jks
│   └── truststore.jks
├── hadoop/
│   ├── core-site.xml
│   └── hdfs-site.xml
└── data/
    ├── var/
    │   └── log/
    │       └── server.log
```

3.2. Konfigürasyon Dosyaları Kategorileri

Kategori	Dosyalar	Görev
Node kimliği	node.properties	Cluster kimliği, node ID, veri dizini
Runtime	jvm.config	JVM flag'leri, heap boyutu, GC ayarları
Cluster davranışı	config.properties	Roller, network, memory, discovery, auth mode
Loglama	log.properties	Modül bazlı log seviyeleri
Kimlik doğrulama	password-authenticator.properties password.db	PASSWORD auth için dosya tabanlı kullanıcı listesi
Yetkilendirme	access-control.properties rules.json	Erişim kontrol kuralları

Kategori	Dosyalar	Görev
Gruplama	group-provider.properties group.txt	Kullanıcı → grup eşleştirmesi
Kaynak yönetimi	resource-groups.properties resource-groups.json	Query kuyrukları, concurrency, memory kotaları
Veri kaynakları	etc/catalog/*.properties	Katalog tanımları (bir dosya = bir katalog)
TLS	keystore.jks, truststore.jks	HTTPS sertifika ve özel anahtar

4. Konfigürasyon Dosyaları — Detaylı Referans

4.1. node.properties

İşlev

Trino process'inin kimlik kartıdır. Her node başlangıçta bu dosyayı okur ve hangi cluster'a ait olduğunu, hangi unique ID ile kayıt olacağını ve verilerini nereye yazacağını buradan öğrenir.

Syntax

Standart Java .properties formatı. Her satır `key=value` şeklinde, yorum satırları `#` ile başlar.

Alan Referansı

Anahtar	Zorunlu	Açıklama
<code>node.environment</code>	Evet	Cluster ismi. Aynı cluster'daki tüm node'larda birebir aynı olmak zorundadır. Küçük harf ve alfanumerik karakterlere izin verilir.
<code>node.id</code>	Evet	Bu node'un cluster içindeki unique kimliği. UUID veya insan-okunur bir string olabilir. İki node aynı ID'yi kullanamaz.
<code>node.data-dir</code>	Evet	Query state, spill dosyaları ve log dosyalarının yazılacağı dizin. Yazılabilir ve kalıcı disk olmalıdır.

Örnek — Tek Instance

```
node.environment=dev
node.id=trino-single
node.data-dir=/data/trino
```

Örnek — Cluster

Coordinator node:

```
node.environment=production
node.id=coord-01
node.data-dir=/data/trino
```

Worker node 1:

```
node.environment=production
node.id=worker-01
node.data-dir=/data/trino
```

Worker node 2:

```
node.environment=production
node.id=worker-02
node.data-dir=/data/trino
```

Kritik Noktalar

- `node.environment` cluster genelinde eşit olmazsa worker'lar coordinator'a register olamaz.
- Ansible ile deploy edilirken `node.id` hostname veya inventory değişkeninden türetilmelidir.

- `node.data-dir` altında `var/log/server.log` asıl detaylı log dosyasıdır (Docker'da `docker logs stdout`'u gösterir ama `server.log` daha kapsamlıdır).

4.2. jvm.config

İşlev

Trino'nun çalışacağı JVM'in başlangıç parametrelerini belirler. Heap boyutu, garbage collector ayarları, hata davranışları ve Java modül izinleri bu dosyada tanımlanır.

Syntax

Diğer konfigürasyon dosyalarının aksine **.properties formatında değildir**. Her satır bir JVM argümanıdır (-Xmx16G gibi). Boş satırlar atlanır, # ile başlayan satırlar yorumdur.

Kritik JVM Flag'leri

Flag	Açıklama
-server	Server-class JIT compiler aktifleştirir. Long-running süreçler için zorunludur.
-Xmx<boyut>	Maksimum heap boyutu. Trino'nun tüm memory hesabı bu değerin üstünden yapılır.
-XX:+ExitOnOutOfMemoryError	OOM durumunda process ölür, zombie kalmaz. Docker restart policy ile toparlanır.
-XX:+HeapDumpOnOutOfMemoryError	OOM anında heap dump yazar. Post-mortem debug için kritik.
-XX:-OmitStackTraceInFastThrow	Kısa exception'larda stack trace atlanmasını engeller. Hata izinde şart.
-Djdk.attach.allowAttachSelf=true	Trino 400+ sürümleri için zorunludur. Olmadan Trino başlamaz.
-Dfile.encoding=UTF-8	Karakter kodlaması. Türkçe verilerin doğru işlenmesi için gereklidir.
-XX:G1HeapRegionSize=32M	G1 garbage collector için heap region boyutu.
-XX:ReservedCodeCacheSize=512M	JIT compile edilmiş kod için ayrılan cache alanı.

Örnek Konfigürasyon

```
- server
-Xmx16G
-XX:InitialRAMPercentage=80
-XX:MaxRAMPercentage=80
-XX:G1HeapRegionSize=32M
-XX:+ExplicitGCInvokesConcurrent
-XX:+HeapDumpOnOutOfMemoryError
-XX:+ExitOnOutOfMemoryError
-XX:-OmitStackTraceInFastThrow
-XX:ReservedCodeCacheSize=512M
-XX:PerMethodRecompilationCutoff=10000
-XX:PerBytecodeRecompilationCutoff=10000
-Djdk.attach.allowAttachSelf=true
-Djdk.nio.maxCachedBufferSize=2000000
-Dfile.encoding=UTF-8
-XX:+UnlockDiagnosticVMOptions
-XX:GCLockerRetryAllocationCount=32
```

Heap Boyutu Hesaplama

Heap boyutu `query.max-memory-per-node + memory.heap-headroom-per-node` toplamından büyük olmak zorundadır. Aşağıdaki formül uygulanmalıdır:

```
-Xmx > query.max-memory-per-node + memory.heap-headroom-per-node
      + (aktif query'lerin toplam memory'si)
```

Örnek: 32GB RAM'li host için

-Xmx24G (host RAM'inin ~%75'i)

query.max-memory-per-node=8GB

memory.heap-headroom-per-node=4GB

→ Kalan ~12GB diğer query'ler ve JVM overhead için

Tek Instance vs Cluster

Ortam	Tipik Heap	Açıklama
Tek instance (dev)	4-8 GB	Küçük veri setleri, tek kullanıcı testleri
Tek instance (prod)	16-32 GB	Orta yük, birkaç eşzamanlı sorgu
Cluster coordinator	24-48 GB	Query planning, metadata cache, UI trafiği ağırlıklı
Cluster worker	64-256 GB	Asıl hesaplama yükü, geniş data buffer'ları

4.3. config.properties

İşlev

Trino cluster'ının davranışsal ayarlarını içerir. Node rolü (coordinator/worker), network portları, memory limitleri, discovery URL, authentication modu ve query kısıtlamaları bu dosyada tanımlanır.

Ana Konfigürasyon Grupları

Node Rolü

Anahtar	Değer	Açıklama
coordinator	true / false	Bu node coordinator mı, worker mı?
node-scheduler.include-coordinator	true / false	Coordinator worker olarak da çalışsın mı? Tek instance için true, cluster'da false.

HTTP/HTTPS

Anahtar	Açıklama
http-server.http.enabled	HTTP servisi açık mı?
http-server.http.port	HTTP dinleme portu (varsayılan 8080)
http-server.https.enabled	HTTPS servisi açık mı? Password auth için zorunlu.
http-server.https.port	HTTPS dinleme portu (varsayılan 8443)
http-server.https.keystore.path	JKS veya PKCS12 keystore dosyasının yolu
http-server.https.keystore.key	Keystore parolası
http-server.process-forwarded	Reverse proxy arkasında X-Forwarded-For header'ı işlensin mi?

Cluster İletişimi

Anahtar	Açıklama
discovery.uri	Coordinator'ın erişilebilir URI'si. Worker'lar bu adrese register olur.
internal-communication.shared-secret	Coordinator ↔ worker arası kimlik doğrulama secret'ı. Tüm node'larda AYNI olmalı. Trino 400+ için zorunludur.
internal-communication.https.required	Internal iletişim HTTPS zorunluluğu (üretim için true)

Kimlik Doğrulama

Anahtar	Açıklama
http-server.authentication.type	PASSWORD, CERTIFICATE, OAUTH2, JWT, KERBEROS, HEADER değerlerinden biri veya virgülle ayrılmış liste
http-server.authentication.allow-insecure-over-http	HTTP üzerinden auth'a izin ver (üretimde false, sadece test için true)

Memory Yönetimi

Anahtar	Açıklama
query.max-memory	Bir query'nin cluster genelinde kullanabileceği toplam memory
query.max-memory-per-node	Bir query'nin tek bir worker node'da kullanabileceği max memory
memory.heap-headroom-per-node	GC ve JVM overhead için ayrılan boşluk. Query'lere verilmez.

Query Limitleri

Anahtar	Açıklama
query.max-execution-time	Bir query'nin execution süresi üst sınırı
query.max-run-time	Queue + execution toplam süre üst sınırı
query.max-history	Web UI'da tutulan finished query sayısı
query.max-length	SQL metninin max karakter sayısı
query.max-stage-count	Query planında oluşabilecek max stage sayısı

Katalog Yönetimi

Anahtar	Açıklama
catalog.management	static (varsayılan) veya dynamic. Dynamic modda CREATE CATALOG SQL komutu ile runtime katalog yönetimi mümkün.
catalog.store	Dynamic katalog dosyalarının tutulduğu dizin

Örnek — Tek Instance Konfigürasyonu

```

coordinator=true
node-scheduler.include-coordinator=true

http-server.http.enabled=true
http-server.http.port=8080

internal-communication.shared-secret=a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2

discovery.uri=http://localhost:8080

query.max-memory=10GB
query.max-memory-per-node=4GB
memory.heap-headroom-per-node=2GB

catalog.management=dynamic
catalog.store=/etc/trino/catalog

```

Örnek — Cluster Coordinator Konfigürasyonu

```

coordinator=true
node-scheduler.include-coordinator=false

```

```

http-server.https.enabled=true
http-server.https.port=8443
http-server.https.keystore.path=/etc/trino/tls/keystore.jks
http-server.https.keystore.key=changeit
http-server.http.enabled=false

http-server.authentication.type=PASSWORD

internal-communication.shared-secret=<uzun-random-hex-string>
internal-communication.https.required=true

discovery.uri=https://trino-coordinator:8443

query.max-memory=200GB
query.max-memory-per-node=16GB
memory.heap-headroom-per-node=4GB

catalog.management=dynamic
catalog.store=/etc/trino/catalog

```

Örnek — Cluster Worker Konfigürasyonu

```

coordinator=false

http-server.https.enabled=true
http-server.https.port=8443
http-server.https.keystore.path=/etc/trino/tls/keystore.jks
http-server.https.keystore.key=changeit
http-server.http.enabled=false

internal-communication.shared-secret=<coordinator-ile-AYNI-secret>
internal-communication.https.required=true

discovery.uri=https://trino-coordinator:8443

query.max-memory-per-node=16GB
memory.heap-headroom-per-node=4GB

catalog.management=dynamic
catalog.store=/etc/trino/catalog

```

Coordinator ve Worker Karşılaştırma Matrisi

Alan	Tek Instance	Coordinator	Worker
coordinator	true	true	false
node-scheduler.include-coordinator	true	false	(yok)
discovery.uri	Kendi adresi	Kendi adresi	Coordinator adresi
authentication.type	Opsiyonel	Set edilir	Set edilmez
query.max-memory	Var	Var	Kullanılmaz
Web UI ayarları	Aktif	Aktif	Kullanılmaz
keystore	Opsiyonel	Zorunlu	Zorunlu (HTTPS)

Alan	Tek Instance	Coordinator	Worker
			internal)

4.4. log.properties

İşlev

Java paket bazında log seviyelerini belirler. Belirli bir modülün detaylı debug çıktısı istenirse veya gereksiz gürültü kesilmek istenirse bu dosya düzenlenir.

Syntax

Standart .properties. Key kısmında Java paket adı, value kısmında log seviyesi (DEBUG, INFO, WARN, ERROR) yer alır.

Örnek

```
io.trino=INFO
io.trino.server=INFO
io.trino.plugin=INFO
io.trino.execution=INFO

# Belirli bir connector'ın debug çıktısı:
io.trino.plugin.hive=DEBUG
io.trino.plugin.postgresql=DEBUG

# Gürültü kaynakları:
com.google.jaxrs=WARN
org.eclipse.jetty=WARN
com.amazonaws=WARN
```

Log Çıktı Konumu

Log çıktıları <node.data-dir>/var/log/server.log dosyasına yazılır. Docker container'da docker logs komutu sadece stdout'u gösterir; asıl detaylı log server.log dosyasındadır.

Tek Instance vs Cluster

Konfigürasyon aynıdır. Cluster'da genellikle coordinator ve worker'larda aynı log.properties kullanılır. Belirli bir worker'da debug açmak gerekirse (örneğin task scheduler sorunları için), o worker'ın log.properties'i geçici olarak DEBUG seviyesine alınır ve worker restart edilir.

4.5. access-control.properties ve rules.json

İşlev

Sistem seviyesi yetkilendirmeyi yönetir. Hangi kullanıcının veya grubun hangi katalog, şema, tablo veya sütuna hangi izinlerle erişebileceğini belirler. Row-level ve column-level security burada tanımlanır.

access-control.properties Syntax

```
access-control.name=file
security.config-file=/etc/trino/rules.json
security.refresh-period=30s
```

access-control.name seçenekleri: allow-all, read-only, file, opa, ranger.

security.refresh-period set edildiğinde rules.json dosyası bu süre içinde otomatik yeniden yüklenir; Trino restart'a gerek kalmaz. Set edilmezse değişiklik için restart gerekir.

rules.json Genel Yapısı

rules.json bir JSON obje'dir ve içinde aşağıdaki bölümler bulunabilir (hepsi opsiyoneldir):

Bölüm	Kontrol Ettiği
catalogs	Katalog seviyesinde erişim (all / read-only / none)
schemas	Şema owner yetkileri (CREATE, DROP, ALTER)
tables	Tablo izinleri (SELECT, INSERT, DELETE, UPDATE, OWNERSHIP), column mask, row filter
functions	Kullanıcı fonksiyonlarının çalıştırılması
procedures	Stored procedure çağrıları
queries	Query görüntüleme ve kill (Web UI, system.runtime)
impersonation	Bir kullanıcının başka kullanıcı adına query çalıştırması
system_information	/v1/info, /v1/cluster gibi management endpoint'ler
system_session_properties	SET SESSION komutuyla değiştirilebilen property'ler
catalog_session_properties	Katalog seviyesi session property'leri
authorization	ALTER ... SET AUTHORIZATION komutu

Kural Değerlendirme Mantığı

- Her bölüm listesi top-to-bottom (yukarıdan aşağı) okunur.
- İlk eşleşen kural uygulanır, sonrakiler değerlendirilmez.
- Bu nedenle en spesifik kurallar önce, catch-all kurallar en sonda yer alır.

Catalog Kuralları

Alanlar

Alan	Tip	Açıklama
user	regex, opsiyonel	Kullanıcı adı deseni. Varsayılan: .*
group	regex, opsiyonel	Grup adı deseni. Kullanıcı bu gruplardan birinde olmalı.
role	regex, opsiyonel	Aktif rol deseni.
catalog	regex, opsiyonel	Katalog adı deseni. Varsayılan: .*
allow	string, ZORUNLU	all, read-only veya none

Örnek

```
"catalogs": [
  { "user": "admin", "catalog": ".*", "allow": "all" },
  { "group": "engineers", "catalog": "(postgres|hive|elasticsearch)", "allow": "all" },
  { "group": "analysts", "catalog": "postgres", "allow": "read-only" },
  { "catalog": "system", "allow": "none" },
  { "user": ".*", "catalog": ".*", "allow": "none" }
]
```

Schema Kuralları

Şema owner yetkilerini kontrol eder. owner: true verilirse kullanıcı o şemayı yaratabilir, silebilir ve yeniden adlandırabilir.

```
"schemas": [
  { "user": "admin", "schema": ".*", "owner": true },
  { "group": "engineers", "catalog": "hive", "schema": "(staging|raw|processed)", "owner": true },
  { "group": "engineers", "catalog": "memory", "schema": ".*", "owner": true },
  { "user": ".*", "schema": "information_schema", "owner": false }
]
```

Table Kuralları — Column ve Row Security

Trino'nun en güçlü yetkilendirme özelliği burada bulunur. Tablo seviyesinde privilege kontrolüne ek olarak sütun bazlı gizleme/maskeleme ve satır filtreleme yapılabilir.

Alanlar

Alan	Tip	Açıklama
user, group, role	regex	Kullanıcı/grup filtresi
catalog, schema, table	regex	Hedef kaynak filtresi
privileges	string array	SELECT, INSERT, DELETE, UPDATE, OWNERSHIP, GRANT_SELECT
columns	obje array	Sütun bazında allow/mask ayarları
columns[].name	string	Sütun adı

Alan	Tip	Açıklama
columns[].allow	boolean	false ise sütun tamamen gizlenir
columns[].mask	SQL expression	Sütun değerini bu SQL ifadesi ile değiştirir
filter	SQL boolean expr	WHERE gibi enjekte edilir; kullanıcı bu satırları görmez
filter_environment.user	string	Filter/mask ifadelerinin hangi user yetkisiyle değerlendirileceği

Örnek — Kapsamlı Kural

```
"tables": [
  {
    "group": "admin",
    "privileges": ["SELECT", "INSERT", "DELETE", "UPDATE", "OWNERSHIP", "GRANT_SELECT"]
  },
  {
    "group": "analysts",
    "catalog": "postgres",
    "schema": "public",
    "table": "users",
    "privileges": ["SELECT"],
    "columns": [
      { "name": "password_hash", "allow": false },
      { "name": "email", "mask": "regexp_replace(email, '(.)*(@.*)', '$1***$2') " },
      { "name": "phone_number", "mask": "'****-***-' || substr(phone_number, -4)" },
      { "name": "salary", "mask": "0" }
    ],
    "filter": "country_code = 'TR'",
    "filter_environment": { "user": "system_filter_user" }
  },
  {
    "group": "engineers",
    "catalog": "(postgres|hive|elasticsearch)",
    "privileges": ["SELECT", "INSERT", "DELETE", "UPDATE"]
  }
]
```

Yukarıdaki örnekte analysts grubundaki bir kullanıcı postgres.public.users tablosundan SELECT yapabilir, ancak:

- password_hash sütununu hiç göremez
- email sütununu maskelenmiş olarak görür (örn. r***@ethem.local)
- phone_number sütununda sadece son 4 hane görünür
- salary sütunu her satırda 0 olarak döner
- Sadece country_code = 'TR' olan satırları görür

Query Kuralları

Query görüntüleme ve iptal etme yetkilerini kontrol eder.

İzin	Etkisi
execute	Query çalıştırma yetkisi

İzin	Etkisi
view	Başkasının query'sini görme (Web UI, system.runtime.queries)
kill	Başkasının query'sini iptal etme

```
"queries": [
  { "user": "admin", "allow": ["execute","view","kill"] },
  { "group": "engineers", "queryOwner": "engineers", "allow": ["execute","view"] },
  { "allow": ["execute"] }
]
```

Impersonation Kuralları

Bir kullanıcının başka kullanıcı adına query çalıştırmasına izin verir. ETHEM benzeri platformlarda backend, tek bir service account ile Trino'ya bağlanıp end-user'ı X-Trino-User header'ı ile geçiriyorsa bu kural zorunludur.

```
"impersonation": [
  { "original_user": "ethem_service", "new_user": ".*", "allow": true },
  { "original_user": "admin", "new_user": ".*", "allow": true }
]
```

System Information Kuralları

/v1/info, /v1/cluster, /v1/node gibi management endpoint'lere erişimi ve graceful shutdown yetkisini kontrol eder.

```
"system_information": [
  { "user": "admin", "allow": ["read","write"] },
  { "group": "engineers", "allow": ["read"] }
]
```

Tek Instance vs Cluster

rules.json sadece coordinator'da bulunur; worker'larda gerekmez. Cluster'da coordinator'daki dosya değişince tüm authorization kararları etkilenir. HA setup'ta (birden fazla coordinator) dosyanın tüm coordinator'larda senkron tutulması gerekir; bunun için config management aracı (Ansible, Kubernetes ConfigMap) veya paylaşımlı filesystem kullanılır.

4.6. password-authenticator.properties ve password.db

İşlev

PASSWORD authentication kullanılacaksa kullanıcı/parola bilgilerini yönetir. LDAP veya OAuth2 kullanılmıyorsa Trino'nun dosya bazlı en basit auth mekanizmasıdır.

password-authenticator.properties

```
password-authenticator.name=file
file.password-file=/etc/trino/password.db
file.refresh-period=5s
file.auth-token-cache-max-size=1000
```

Anahtar	Açıklama
password-authenticator.name	Authenticator tipi: file, ldap veya özel plugin
file.password-file	Kullanıcı dosyasının yolu
file.refresh-period	Dosya değişikliği kontrol sıklığı (restart'sız güncelleme)
file.auth-token-cache-max-size	Bcrypt hesaplaması pahalıdır; başarılı auth token'ları cache'lenir

Çoklu Authenticator

Birden fazla authenticator sırayla denenebilir. Örneğin önce LDAP, LDAP başarısız olursa dosya:

```
# config.properties
http-server.authentication.type=PASSWORD
password-authenticator.config-files=etc/ldap.properties,etc/file.properties
```

password.db Formatı

Her satırda bir kullanıcı, format:

```
username:hashed_password
```

Bcrypt Formatı (Önerilen)

Hash \$2y\$ ile başlar, cost minimum 8 olmalıdır. Trino sadece \$2y\$ prefix'ini kabul eder; \$2a\$ ve \$2b\$ reddedilir.

```
admin:$2y$10$BqTb8hScP5DfcpmHo5PeyugxHz5Ky/qf3wrpD7SNm8sWuA3VlGqsa
ragnar:$2y$10$K7XjZuG8LhVPvNwXqYsMe.QRtNvHzMxZKcJpG9tNzKvJdW3xR8m2u
analyst1:$2y$10$cN3xR8m2uK7XjZuG8LhVPvNwXqYsMe.QRtNvHzMxZKcJpG9tNzK
```

Bcrypt Hash Üretimi

```
# Yeni dosya oluşturma + ilk kullanıcı
htpasswd -B -C 10 -c /etc/trino/password.db admin

# Mevcut dosyaya kullanıcı ekleme (-c YOK)
htpasswd -B -C 10 /etc/trino/password.db ragnar
htpasswd -B -C 10 /etc/trino/password.db analyst1
```

```
# Parametreler:  
# -B = bcrypt kullan  
# -C = cost (10 önerilen, minimum 8)  
# -c = yeni dosya oluştur (dikkat: mevcut dosyayı üzerine yazar)
```

PBKDF2 Formatı (Alternatif)

```
username:iterations:hex_salt:hex_hash  
  
# Örnek:  
admin:1000:5b4240333032306164:f38d165fce8ce42f59d366139ef5d9e1ca1247f0e06e503ee1a61  
1dd
```

Tek Instance vs Cluster

password.db sadece coordinator'da bulunur; worker'lar client kimlik doğrulaması yapmaz. HA cluster'da (multi-coordinator) dosyanın tüm coordinator'larda birebir aynı olması için config management ile push edilmesi veya paylaşımlı dosya sistemi (NFS, ConfigMap) kullanılması gerekir.

4.7. group-provider.properties ve group.txt

İşlev

Kullanıcı → grup eşleştirmesini sağlar. rules.json ve resource-groups.json dosyalarındaki group filtreleri bu dosyaya bakar. Kullanıcı-grup ilişkisi olmadan grup bazlı kurallar çalışmaz.

group-provider.properties

```
group-provider.name=file
file.group-file=/etc/trino/group.txt
file.refresh-period=30s
```

Provider tipleri: file, ldap (Enterprise), veya özel plugin.

group.txt Formatı

```
grup_adi:user1,user2,user3
```

Her satır bir grubu tanımlar. Grup adı iki nokta üst üste ile ayrılır, kullanıcı listesi virgülle. Boşluk kullanılmaz.

Örnek

```
# Yönetici grubu
admins:admin,ragnar

# Veri mühendisleri
engineers:ragnar,ahmet,mehmet

# Analiz ekibi
analysts:ayse,fatma,zeynep

# Salt okuma erişimi
readonly:guest,demo

# Proje grupları – bir kullanıcı birden fazla grupta olabilir
osint_team:ragnar,ahmet,ayse
etl_team:ahmet,mehmet
```

Önemli Notlar

- Bir kullanıcı birden fazla grupta yer alabilir (yukarıdaki örnekte ragnar üç grupta).
- Kullanıcı listesinde boşluk olmamalı; user1, user2 değil, user1,user2 şeklinde.
- Yorum satırları # ile başlar.

Tek Instance vs Cluster

Sadece coordinator'da bulunur. Cluster'da grup değişiklikleri sadece coordinator'a etki eder çünkü authorization kararları orada verilir.

4.8. resource-groups.properties ve resource-groups.json

İşlev

Query'lerin hangi kuyrukta çalışacağını, ne kadar memory ve CPU kullanabileceğini, aynı anda kaç query'nin çalışabileceğini yönetir. Cluster kaynaklarını farklı iş yükleri arasında adil dağıtır ve QoS (Quality of Service) sağlar.

resource-groups.properties

Dosya Tabanlı Manager

```
resource-groups.configuration-manager=file
resource-groups.config-file=/etc/trino/resource-groups.json
```

Veritabanı Tabanlı Manager (Üretim İçin Önerilen)

```
resource-groups.configuration-manager=db
resource-groups.config-db-url=jdbc:postgresql://pg:5432/resource_groups
resource-groups.config-db-user=trino_rg
resource-groups.config-db-password=xxx
resource-groups.exact-match-selector-enabled=true
```

DB modu şu avantajları sağlar: konfigürasyon değişikliği için restart gerekmez, HA cluster'da (multi-coordinator) otomatik senkron sağlanır, değişiklikler audit'lenebilir. Desteklenen veritabanları: MySQL, PostgreSQL, Oracle. Tablolar (resource_groups, selectors, resource_groups_global_properties) otomatik yaratılır.

resource-groups.json Yapısı

İki ana bölümden oluşur:

- **rootGroups** — Grup hiyerarşisi (ağaç yapısı). Her grup memory limiti, concurrency limiti ve alt gruplar içerir.
- **selectors** — Query → grup yönlendirme kuralları. Kullanıcı, kaynak, query tipi gibi kriterlere göre query'yi doğru gruba atar.

Group Alanları

Alan	Zorunlu	Açıklama
name	Evet	Grup adı. \${USER}, \${SOURCE} template değişkenleri desteklenir.
softMemoryLimit	Evet	Bu limit aşıldığında yeni query'ler kuyruğa alınır. Yüzde (80%) veya mutlak (10GB).
hardConcurrencyLimit	Evet	Aynı anda çalışabilen max query sayısı.
maxQueued	Evet	Max kuyruk boyutu. Aşırsa yeni query REJECT edilir.
softConcurrencyLimit	Hayır	Yumuşak concurrency limiti. Peer group'lar boşsa aşılabılır.
schedulingPolicy	Hayır	fair (varsayılan), weighted, weighted_fair, query_priority
schedulingWeight	Hayır	Weighted policy'de öncelik değeri (yüksek = daha fazla pay)
softCpuLimit	Hayır	Bu kotayı aşan query'ler yavaşlatılır (cpuQuotaPeriod içinde)

Alan	Zorunlu	Açıklama
hardCpuLimit	Hayır	Bu kotayı aşan query'ler kill'lenir
jmxExport	Hayır	true ise grup metrikleri JMX'e yayınlanır
subGroups	Hayır	Alt grup listesi. Hiyerarşik yapı için.

Selector Alanları

Alan	Açıklama
user	Kullanıcı adı regex'i
originalUser	Impersonation öncesi kullanıcı
authenticatedUser	Auth katmanının döndürdüğü user
userGroup	Kullanıcının bulunduğu grup regex'i
source	Query kaynağı (--source CLI flag, JDBC source property)
queryType	SELECT, INSERT, DELETE, DATA_DEFINITION, EXPLAIN, DESCRIBE, ANALYZE
clientTags	Client tag listesi (superset match yapılır)
clientInfo	Client info regex'i
queryText	Query metni regex'i
group	ZORUNLU. Query'nin gideceği hedef grup.

Selector alanları AND ile birleştirilir (hepsi eşleşmeli). Selector'lar top-to-bottom taranır, ilk eşleşen kazanır. Hiçbir selector eşleşmezse query reject edilir.

Kapsamlı Örnek

```
{
  "rootGroups": [
    {
      "name": "global",
      "softMemoryLimit": "80%",
      "hardConcurrencyLimit": 100,
      "maxQueued": 1000,
      "schedulingPolicy": "weighted_fair",
      "jmxExport": true,
      "subGroups": [
        {
          "name": "admin",
          "softMemoryLimit": "40%",
          "hardConcurrencyLimit": 30,
          "maxQueued": 100,
          "schedulingWeight": 200
        },
        {
          "name": "etl",
          "softMemoryLimit": "50%",
          "hardConcurrencyLimit": 40,
          "maxQueued": 500,
          "schedulingWeight": 100,

```

```

    "subGroups": [
      {
        "name": "user_${USER}",
        "softMemoryLimit": "20%",
        "hardConcurrencyLimit": 10,
        "maxQueued": 100
      }
    ],
    {
      "name": "adhoc",
      "softMemoryLimit": "30%",
      "hardConcurrencyLimit": 20,
      "maxQueued": 200,
      "schedulingWeight": 50,
      "subGroups": [
        {
          "name": "user_${USER}",
          "softMemoryLimit": "10%",
          "hardConcurrencyLimit": 3,
          "maxQueued": 20,
          "softCpuLimit": "30m",
          "hardCpuLimit": "1h"
        }
      ]
    }
  ],
  "selectors": [
    { "group": "admins", "group": "global.admin" },
    { "group": "engineers", "source": ".*(airflow|etl|pipeline).*",
      "group": "global.etl.user_${USER}" },
    { "group": "engineers", "group": "global.adhoc.user_${USER}" },
    { "group": "global.adhoc.user_${USER}" }
  ],
  "cpuQuotaPeriod": "1h"
}

```

Template Değişkenleri

`${USER}` ve `${SOURCE}` template değişkenleri, dinamik olarak per-user veya per-source alt gruplar oluşturur. Örneğin `user_${USER}` isimli grup, `ragnar` sorgu attığında `user_ragnar`, `ahmet` sorgu attığında `user_ahmet` olarak somutlanır. Bu, her kullanıcının kendi kotası olmasını sağlar.

Tek Instance vs Cluster

Tek instance'ta `softMemoryLimit` değerleri tek node'un memory'sinin yüzdesi olarak yorumlanır. Cluster'da ise cluster'ın toplam memory'sinin yüzdesidir. Yani 5-worker'lı ve her worker'ı 32GB'lık bir cluster'da `softMemoryLimit: 50%` değeri ~80GB memory'ye karşılık gelir. Sadece coordinator'da bulunur; worker'lar resource group kararına katılmaz.

4.9. Katalog Konfigürasyonları

İşlev

Her .properties dosyası bir katalog tanımlar ve **dosya adı katalog adı olarak kullanılır**. Örneğin postgres.properties → SQL'de postgres.public.users şeklinde erişilebilen postgres katalogunu tanımlar.

Genel Syntax

```
connector.name=<connector-tipi>          # Zorunlu, ilk satır
<connector-özel-anahtar>=<değer>
<connector-özel-anahtar>=<değer>
```

connector.name her katalog dosyasında ilk satır olarak yer alır ve zorunludur. Bir kez set edildikten sonra değiştirilemez; değiştirilmek istenirse dosya silinip yeniden oluşturulmalıdır.

PostgreSQL Katalog

```
connector.name=postgresql
connection-url=jdbc:postgresql://pg-host:5432/mydb
connection-user=trino_ro
connection-password=StrongPassword

case-insensitive-name-matching=true
join-pushdown.enabled=true
join-pushdown.strategy=EAGER
decimal-mapping=allow_overflow
decimal-rounding-mode=HALF_UP
unsupported-type-handling=CONVERT_TO_VARCHAR

# Metadata cache (sık tablo değişmeyen ortamlarda hız verir)
metadata.cache-ttl=1h
metadata.cache-missing=true
```

Hive / HDFS Katalog

```
connector.name=hive
hive.metastore.uri=thrift://hms:9083

# HDFS konfigürasyon dosyaları – container'a mount edilmeli
hive.config.resources=/etc/trino/hadoop/core-site.xml,/etc/trino/hadoop/hdfs-site.xml

hive.non-managed-table-writes-enabled=true
hive.allow-drop-table=true
hive.allow-rename-table=true
hive.allow-add-column=true
hive.allow-drop-column=true

# Performance
hive.metastore-cache-ttl=1h
hive.parquet.use-column-names=true

# Kerberos (opsiyonel)
# hive.metastore.authentication.type=KERBEROS
```

```
# hive.metastore.service.principal=hive/_HOST@REALM
# hive.metastore.client.principal=trino/_HOST@REALM
# hive.metastore.client.keytab=/etc/trino/trino.keytab
```

Elasticsearch Katalog

```
connector.name=elasticsearch
elasticsearch.host=es-host
elasticsearch.port=9200
elasticsearch.default-schema-name=default

# Authentication
elasticsearch.security=PASSWORD
elasticsearch.auth.user=elastic
elasticsearch.auth.password=elasticPassword

# TLS
elasticsearch.tls.enabled=true
elasticsearch.tls.truststore-path=/etc/trino/tls/es-truststore.jks
elasticsearch.tls.truststore-password=changeit
elasticsearch.tls.verify-hostnames=false

# Timeout ayarları
elasticsearch.request-timeout=30s
elasticsearch.connect-timeout=10s
elasticsearch.max-http-connections=25
```

MySQL Katalog

```
connector.name=mysql
connection-url=jdbc:mysql://mysql-host:3306
connection-user=trino_user
connection-password=mysqlPassword

case-insensitive-name-matching=true
join-pushdown.enabled=true
```

MinIO / S3 (Iceberg) Katalog

```
connector.name=iceberg
iceberg.catalog.type=hive_metastore
hive.metastore.uri=thrift://hms:9083

fs.native-s3.enabled=true
s3.endpoint=http://minio:9000
s3.region=us-east-1
s3.path-style-access=true
s3.aws-access-key=minioadmin
s3.aws-secret-key=minioadmin
```

Kafka Katalog

```
connector.name=kafka
kafka.nodes=kafka1:9092,kafka2:9092,kafka3:9092
kafka.table-names=events,logs,audit
```

```
kafka.hide-internal-columns=false
kafka.default-schema=default
```

Memory Katalog (Test İçin)

```
connector.name=memory
memory.max-data-per-node=512MB
```

Memory connector volatile'dir — Trino restart'ında veri kaybolur. Test, CTAS ve geçici hesaplama için idealdir.

Catalog-Level Access Control

Sistem seviyesi `rules.json` yerine (veya buna ek olarak) tek bir katalog için ayrı kural dosyası tanımlanabilir. Bu, modüler ETHER benzeri platformlarda tercih edilir.

```
# hive.properties içine eklenir:
hive.security=file
security.config-file=/etc/trino/catalog/hive-rules.json
```

`hive-rules.json` sadece o katalog scope'unda çalışır ve `schemas`, `tables`, `session_properties` bloklarını içerir (catalogs bloğu yoktur, çünkü katalog seviyesindedir).

Secret Yönetimi

Katalog dosyalarında parola gibi hassas bilgilerin düz metin bırakılması güvenlik açığıdır. Environment variable referansı kullanılmalıdır:

```
connection-password=${ENV:POSTGRES_TRINO_PASSWORD}
elasticsearch.auth.password=${ENV:ELASTIC_PASSWORD}
s3.aws-secret-key=${ENV:MINIO_SECRET_KEY}
```

Docker Compose'da environment variable'lar container'a şu şekilde geçirilir:

```
services:
  trino:
    image: trinodb/trino:481
    environment:
      POSTGRES_TRINO_PASSWORD: ${POSTGRES_TRINO_PASSWORD}
      ELASTIC_PASSWORD: ${ELASTIC_PASSWORD}
      MINIO_SECRET_KEY: ${MINIO_SECRET_KEY}
```

Host makinede `.env` dosyası:

```
POSTGRES_TRINO_PASSWORD=StrongPassword123
ELASTIC_PASSWORD=elasticPassword
MINIO_SECRET_KEY=minioSecretKey123
```

`.env` dosyası mutlaka `.gitignore`'a eklenmeli, versiyon kontrolüne dahil edilmemelidir.

Static ve Dynamic Katalog Yönetimi

Static Mod (Varsayılan)

Katalog dosyaları `/etc/trino/catalog/` altına elle konulur ve Trino restart edilir. Değişiklik için her seferinde restart gerekir. Downtime yaratır.

Dynamic Mod

config.properties'e catalog.management=dynamic eklenir. Sonrasında SQL komutlarıyla runtime'da katalog eklenip çıkarılabilir:

```
CREATE CATALOG postgres USING postgresql
WITH (
  "connection-url" = 'jdbc:postgresql://pg:5432/db',
  "connection-user" = 'user',
  "connection-password" = 'pass'
);

DROP CATALOG postgres;
SHOW CATALOGS;
```

Dynamic mod restart'sız katalog eklemek için idealdir. ETHEM gibi modüler platformlarda mail, phone, HDFS gibi modüllerin runtime'da bağlanması bu modda gerçekleşir.

Tek Instance vs Cluster

Tek instance'ta tüm .properties dosyaları tek /etc/trino/catalog/ dizininde bulunur ve doğrudan kullanılır.

Cluster'da **tüm node'larda (coordinator ve tüm worker'lar) aynı katalog dosyalarının bulunması ZORUNLUDUR**. Bir worker'da eksik katalog varsa, o worker'a düşen split fail olur. Çözüm yolları:

- Config management (Ansible, Puppet, Chef): dosyaları tüm node'lara push et
- Shared filesystem (NFS): /etc/trino/catalog dizinini paylaşımlı disk olarak mount et
- Kubernetes ConfigMap: ConfigMap'i tüm pod'lara mount et

4.10. TLS Sertifikaları ve Keystore Yönetimi

İşlev

HTTPS iletişimi için gereken sertifika ve özel anahtar dosyalarıdır. Password authentication kullanılacaksa HTTPS zorunludur; bu nedenle keystore da zorunlu hale gelir.

Desteklenen Formatlar

Format	Açıklama	Kullanım
JKS	Klasik Java KeyStore formatı	Java-only ortamlar
PKCS12 (.p12)	Modern cross-platform standart	Yeni kurulumlar
PEM	Text-based (Base64)	Direkt kullanılmaz; JKS/PKCS12'ye dönüştürülür

Self-Signed Keystore Üretimi

```
keytool -genkeypair \
  -alias trino \
  -keyalg RSA \
  -keysize 2048 \
  -validity 3650 \
  -keystore keystore.jks \
  -storepass changeit \
  -keypass changeit \
  -dname "CN=trino.example.com, OU=Data, O=Corp, C=TR" \
  -ext
SAN=dns:trino.example.com,dns:coordinator,dns:localhost,ip:10.0.0.10,ip:127.0.0.1
```

SAN (Subject Alternative Name) alanı doğru doldurulmalıdır. Sertifikanın hostname'i, client'ın bağlandığı adresle eşleşmezse TLS handshake başarısız olur ve bağlantı kurulamaz.

Truststore Üretimi

```
# Sertifikayı export et
keytool -export -alias trino -keystore keystore.jks \
  -storepass changeit -file trino.crt

# Truststore'a import et (client'lar için)
keytool -import -alias trino -file trino.crt \
  -keystore truststore.jks -storepass changeit -noprompt
```

PEM'den JKS'e Dönüşüm (Real Cert için)

```
# PEM → PKCS12
openssl pkcs12 -export \
  -inkey privkey.pem -in fullchain.pem \
  -out keystore.p12 -name trino -password pass:changeit

# PKCS12 → JKS
keytool -importkeystore \
  -srckeystore keystore.p12 -srcstoretype PKCS12 \
  -srcstorepass changeit \
```

```
-destkeystore keystore.jks -deststorepass changeit
```

config.properties Referansı

```
http-server.https.enabled=true
http-server.https.port=8443
http-server.https.keystore.path=/etc/trino/tls/keystore.jks
http-server.https.keystore.key=changeit
http-server.http.enabled=false

# Internal communication de HTTPS ise
internal-communication.https.required=true
internal-communication.https.keystore.path=/etc/trino/tls/internal-keystore.jks
internal-communication.https.keystore.key=xxx
```

Tek Instance vs Cluster

Tek instance'ta bir keystore yeterlidir.

Cluster'da iki yaklaşım vardır:

- Her node'un kendi sertifikası olur (hostname doğru match olur, güvenlik seviyesi yüksek). Sertifika yönetimi karmaşıklaşır.
- Wildcard sertifika (*.trino.internal) veya SAN'a tüm hostname'leri koyulmuş tek sertifika tüm node'larda paylaşılır. Daha basit ama tek noktadan compromise riski.

5. Trino API Endpoint Referansı

Coordinator'un tüm REST API endpoint'leri `https://coordinator:8443` altında yer alır. Web UI, CLI, JDBC driver ve diğer client'lar bu API üzerinden çalışır. Programatik entegrasyon veya monitoring için doğrudan çağrılabilir.

5.1. Query Yönetim Endpoint'leri

Endpoint	Metod	İşlev
/v1/statement	POST	Ana query submission endpoint'i. Body: SQL text. Response: nextUri, id, stats, columns, data.
/v1/statement/{queryId}/{token}	GET	Sonuçların sayfa sayfa okunması (nextUri takibi).
/v1/statement/{queryId}/{token}	DELETE	Çalışan query'nin iptali.
/v1/query	GET	Tüm query'lerin listesi (Web UI'nin kullandığı endpoint).
/v1/query/{queryId}	GET	Bir query'nin tam detayı (plan, stages, tasks, splits).
/v1/query/{queryId}	DELETE	Query kill.

5.2. Cluster ve Node Yönetimi

Endpoint	Metod	İşlev
/v1/cluster	GET	Cluster durumu, aktif node sayısı, running query sayısı.
/v1/node	GET	Worker node listesi ve durumları.
/v1/node/failed	GET	Fail durumdaki node'lar.
/v1/info	GET	Coordinator version, environment, uptime, starting flag.
/v1/info/state	GET/PUT	ACTIVE / SHUTTING_DOWN. Graceful shutdown için PUT ile "SHUTTING_DOWN".
/v1/info/coordinator	GET	Bu node coordinator mı? Boolean.
/v1/status	GET	Node health/status.
/v1/memory	GET	Cluster memory pool durumu.

5.3. Task ve Diagnostic

Endpoint	Metod	İşlev
/v1/task	GET	Task listesi (internal, coordinator ↔ worker iletişimi).
/v1/task/{taskId}	GET	Task detayı.
/v1/thread	GET	JVM thread dump.
/v1/jmx/mbean	GET	JMX üzerinden metrik ve mbean erişimi.

5.4. UI Endpoint'leri

Endpoint	Metod	İşlev
/ui/	GET	Web UI ana sayfası.
/ui/api/*	GET	Web UI'nin backend API çağrıları.
/ui/login	POST	Web UI login formu submit.
/ui/logout	GET	Logout.

5.5. Query Gönderme Akışı

Trino'da bir query şu adımlarla çalıştırılır:

- Client, /v1/statement endpoint'ine POST ile SQL gönderir.

Response'ta nextUri döner. Bu URI takip edilerek query'nin tamamlanması beklenir.

Her nextUri çağrısında hem sonuç chunk'ları hem de yeni bir nextUri döner. Tüm veriler alınana kadar bu döngü sürer.

nextUri gelmezse query bitmiştir (state: FINISHED veya FAILED).

5.6. HTTP Header'lar

Header	İşlev
Authorization	Basic Auth (password auth aktifse zorunlu)
X-Trino-User	Query sahibi kullanıcı adı
X-Trino-Catalog	Varsayılan katalog
X-Trino-Schema	Varsayılan şema
X-Trino-Source	Query kaynağı (resource group selector'da kullanılır)
X-Trino-Client-Tags	Client tag listesi (virgülle ayrılmış)
X-Trino-Session	Session property'ler

5.7. Örnek — Curl ile Query

```
# Query gönder
curl -X POST http://localhost:8085/v1/statement \
  -H "X-Trino-User: admin" \
  -H "X-Trino-Catalog: postgres" \
  -H "X-Trino-Schema: public" \
  -H "Content-Type: text/plain" \
  -d "SELECT * FROM users LIMIT 10"

# Response içindeki nextUri'yi takip et
curl "http://localhost:8085/v1/statement/executing/{queryId}/.../{token}"
```

5.8. Örnek — Python ile Query

```
from trino.dbapi import connect
from trino.auth import BasicAuthentication

conn = connect(
    host="coordinator.example.com",
    port=8443,
    user="ragnar",
    catalog="postgres",
    schema="public",
    http_scheme="https",
    auth=BasicAuthentication("ragnar", "password"),
    verify=True, # veya CA cert path
)

cur = conn.cursor()
cur.execute("SELECT id, name FROM users WHERE country = ?", ("TR",))
for row in cur.fetchall():
    print(row)
```

6. Web UI ve Yönetim Arayüzü

6.1. Erişim

Web UI, coordinator'un ana portundan erişilir: <https://coordinator:8443/ui/>. Password auth aktifse login ekranı açılır; aksi halde doğrudan dashboard'a girilir.

6.2. Ana Sayfa — Cluster Overview

Üst kısımda gerçek zamanlı güncellenen cluster durum kartları bulunur:

Metrik	Açıklama
Running Queries	Şu an çalışan query sayısı
Queued Queries	Resource group limitine takılıp kuyrukta bekleyen query sayısı
Blocked Queries	Kaynak veya split bekleyen query sayısı
Active Workers	Cluster'daki aktif worker node sayısı
Runnable Drivers	Driver seviyesinde parallelism göstergesi
Reserved Memory	Cluster genelinde kullanılan memory
Rows/sec, Bytes/sec	Throughput
Worker Parallelism	CPU kullanım yoğunluğu

6.3. Query Listesi ve Detayı

Ana sayfanın gövdesinde tüm query'ler listelenir. Her satırda Query ID, user, source, resource group, state, elapsed time, memory ve CPU bilgileri yer alır.

Bir query'e tıklandığında detay sayfası açılır. Bu sayfa Trino'nun en güçlü debug aracıdır:

- **Overview** — Tam SQL, session info, zamanlama, resource usage, error detail
- **Live Plan** — Execution plan görsel gösterimi, her operator'ün input/output row sayısı, predicate pushdown durumu
- **Stage Performance** — Stage bazlı skew analizi, hangi task ne kadar veri işledi, buffer utilization, spill miktarı
- **Splits** — Query'nin böldüğü tüm split'ler, hangi worker'da çalıştı, süre — stragglers split tespiti için kritik
- **JSON** — Query'nin tüm metadata'sı ham JSON — otomasyon için kullanışlı

6.4. Worker İzleme

Sol menüden Workers sayfası açılır. Node listesi (nodeld, IP, HTTP URI, version), her worker için heap kullanımı, non-heap memory, processor count, uptime bilgileri görüntülenir. Bir worker'a tıklandığında JVM stats, GC istatistikleri, thread count ve memory pool detayı görülür.

6.5. Web UI'da Yapılamayanlar

Web UI kasıtlı olarak read-only observability aracı olarak tasarlanmıştır. Aşağıdaki işlemler UI'da bulunmaz:

- Katalog ekleme veya silme (dosya veya SQL ile yapılır)
- Kullanıcı ekleme (password.db düzenlenir)
- Yetki değiştirme (rules.json düzenlenir)
- Resource group değişikliği (resource-groups.json veya DB tablosu)
- Ad-hoc query çalıştırma (SQL editörü yok; CLI, JDBC veya API kullanılır)
- Cluster ayarlarını değiştirme (config.properties + restart)

7. Tek Instance ve Cluster Kurulumları — Karşılaştırma

7.1. Mimari Farklar

Tek Instance (Single Node)

Tek bir Trino process'i hem coordinator hem worker rolünü üstlenir. Query planning, execution, veri erişimi hepsi aynı JVM'de gerçekleşir. Kurulum basittir; ancak ölçeklenebilirlik ve fault tolerance yoktur.

Kullanım Alanları

- Geliştirme ve test ortamları
- PoC (Proof of Concept) çalışmaları
- Tek kullanıcı ad-hoc analiz
- CI/CD pipeline'larında Trino integration testleri

Avantajları

- Basit kurulum, tek konfigürasyon seti
- Minimum kaynak gereksinimi
- Debug ve troubleshooting kolay

Kısıtları

- Ölçeklenebilirlik yok — memory ve CPU tek node ile sınırlı
- High availability yok — node down = servis down
- Concurrent query kapasitesi düşük

Cluster (Multi Node)

Bir coordinator ve N adet worker'dan oluşur. Coordinator query planlarını üretir ve worker'lara dağıtır; worker'lar asıl hesaplama yükünü paylaşır. Discovery servisi worker'ların coordinator'a otomatik register olmasını sağlar.

Kullanım Alanları

- Üretim ortamları
- Petabyte ölçeğinde veri sorgulama
- Çoklu kullanıcı ve iş yükü (ETL + BI + ad-hoc)
- 7/24 servis gereksinimi

Avantajları

- Yatay ölçekleme — worker eklemek anlık kapasite artışı sağlar
- Bir worker down olsa da diğerleri sorguya devam eder
- Yüksek concurrent query kapasitesi
- Resource groups ile iş yükleri izole edilir

Kısıtları

- Coordinator SPOF (Single Point of Failure) — Trino Gateway ile HA yapılır

- Konfigürasyon yönetimi karmaşık — tüm node'larda senkron dosyalar
- Network trafiği büyük (shuffle, exchange) — yüksek bant genişliği ister

7.2. Konfigürasyon Farkları

Alan	Tek Instance	Cluster Coordinator	Cluster Worker
coordinator	true	true	false
node-scheduler.include-coordinator	true	false	(yok)
discovery.uri	Kendi adresi	Kendi adresi	Coordinator adresi
authentication.type	Opsiyonel	Zorunlu	Yok
query.max-memory	Set edilir	Set edilir	Yok
query.max-memory-per-node	Set edilir	Set edilir	Set edilir
rules.json	Var	Var	Yok
password.db	Var	Var	Yok
group.txt	Var	Var	Yok
resource-groups.json	Var	Var	Yok
Katalog dosyaları	Var	Var	Var (senkron)
TLS keystore	Opsiyonel	Zorunlu	Zorunlu
internal-communication.shared-secret	Var (opsiyonel)	Var (AYNI)	Var (AYNI)

7.3. Kaynak Planlama

Parametre	Tek Instance	Cluster
RAM (min)	8 GB	Coordinator: 32 GB, her worker: 64 GB
CPU (min)	4 core	Coordinator: 8 core, her worker: 16 core
Disk (spill için)	50 GB SSD	Her node: 500 GB SSD
Network	1 Gbps	10+ Gbps (shuffle trafiği yoğun)
Concurrent query	5-10	50-500 (worker sayısına bağlı)

7.4. Deployment ve Yönetim

Alan	Tek Instance	Cluster
Deployment	Docker run veya tek binary	Docker Compose, Kubernetes, Ansible
Config güncelleme	Tek dosya, tek restart	Config management ile tüm node'lar
Katalog ekleme	Tek dizin	Tüm node'lar senkron (veya dynamic mode)

Alan	Tek Instance	Cluster
Monitoring	Web UI yeter	Prometheus + Grafana + JMX exporter
Backup	Sadece config	Config + resource-groups DB
Log agregasyonu	Docker logs yeter	Fluentd/Filebeat + Elasticsearch

8. Kurulum Adımları ve Sıralı Uygulama

Sıfırdan güvenli bir Trino kurulumu için önerilen sıra aşağıdadır. Her adımdan sonra test edip bir sonrakine geçilmelidir. Bu sıra hem tek instance hem cluster için geçerlidir; cluster'da her adım tüm node'larda uygulanır.

8.1. Faz 1 — Temel Kurulum (Insecure Mode)

- **Adım 1:** Dizin yapısını oluştur (/opt/trino/etc, catalog, tls, data, hadoop)
- **Adım 2:** node.properties, jvm.config, log.properties dosyalarını yaz
- **Adım 3:** config.properties'i temel (HTTP, auth yok) modda yaz. catalog.management=dynamic ekle.
- **Adım 4:** Docker container'ı volume mount ile başlat, /v1/info endpoint'inden servisi doğrula
- **Adım 5:** İlk katalog dosyalarını (memory, postgres) ekle, SHOW CATALOGS ile doğrula

8.2. Faz 2 — TLS ve Password Authentication

- **Adım 6:** TLS keystore üret (self-signed veya real cert), /opt/trino/tls'e kopyala
- **Adım 7:** password.db oluştur (htpasswd -B -C 10), admin ve test kullanıcılarını ekle
- **Adım 8:** password-authenticator.properties yaz
- **Adım 9:** config.properties'te HTTPS aç, HTTP kapat, authentication.type=PASSWORD ekle, restart
- **Adım 10:** Web UI login test et, CLI ile --password ile bağlan

8.3. Faz 3 — Yetkilendirme

- **Adım 11:** group.txt yaz (admins, engineers, analysts, readonly)
- **Adım 12:** group-provider.properties yaz
- **Adım 13:** rules.json'u laksçı kurallarla başlat (admin'e all, herkese read-only). Test et.
- **Adım 14:** access-control.properties yaz, restart, farklı kullanıcılarla test et
- **Adım 15:** Kuralları sıkılaştır (grup bazlı, catalog bazlı, catch-all deny)
- **Adım 16:** Column mask ve row filter kurallarını ekle, analyst kullanıcısıyla test et

8.4. Faz 4 — Kaynak Yönetimi

- **Adım 17:** resource-groups.json yaz (global, admin, etl, adhoc grupları)
- **Adım 18:** resource-groups.properties yaz (dev'de file, prod'da db manager)
- **Adım 19:** Farklı source ile query at, system.runtime.queries'ten resource_group_id'yi doğrula

8.5. Faz 5 — Katalog ve Entegrasyon

- **Adım 20:** Tüm production kataloglarını ekle (PostgreSQL, Hive, Elasticsearch, MinIO, Kafka)
- **Adım 21:** Katalog parolalarını \${ENV:...} sözdizimiyle environment variable'a taşı
- **Adım 22:** Docker Compose'a .env dosyası referansı ekle
- **Adım 23:** Her katalog için SHOW SCHEMAS ve basit SELECT çalıştırarak bağlantıyı doğrula

8.6. Faz 6 — Cluster'a Ölçekleme (Opsiyonel)

- **Adım 24:** Coordinator config.properties'i node-scheduler.include-coordinator=false yap
- **Adım 25:** İlk worker node'unu hazırla (ayrı node.properties, coordinator=false config.properties)
- **Adım 26:** internal-communication.shared-secret'i tüm node'larda aynı yap
- **Adım 27:** Katalog dosyalarını worker'a senkron et (Ansible, NFS, veya dynamic mode)
- **Adım 28:** Worker'ı başlat, /v1/node endpoint'inden worker'ın kaydolduğunu doğrula
- **Adım 29:** İhtiyaç kadar worker ekle

8.7. Faz 7 — İzleme ve Audit (Prod)

- **Adım 30:** event-listener.properties ekle (Kafka veya HTTP event sink)
- **Adım 31:** JMX exporter ile Prometheus/Grafana entegrasyonu
- **Adım 32:** Log agregasyon pipeline'ı (Filebeat/Fluentd → Elasticsearch)
- **Adım 33:** Backup planı (config repo + resource-groups DB dump)

9. Doğrulama, Test ve İzleme

9.1. Servis Sağlığı

```
# Coordinator info endpoint
curl http://localhost:8085/v1/info | jq

# Cluster durumu
curl http://localhost:8085/v1/cluster | jq

# Node listesi
curl http://localhost:8085/v1/node | jq
```

9.2. Katalog Testleri

```
# Kataloglar yüklendi mi?
docker exec -it trino-secure-trino \
  trino --user admin --execute "SHOW CATALOGS"

# Şemalar erişilebilir mi?
docker exec -it trino-secure-trino \
  trino --user admin --execute "SHOW SCHEMAS FROM postgres"

# Basit sorgu
docker exec -it trino-secure-trino \
  trino --user admin --execute "SELECT * FROM postgres.public.users LIMIT 1"
```

9.3. Authentication Testi

```
# HTTPS + password ile bağlanma
trino --server https://localhost:8443 \
  --user ragnar --password \
  --truststore-path /path/to/truststore.jks \
  --truststore-password changeit

# Yanlış parola reject edilmeli
trino --server https://localhost:8443 \
  --user ragnar --password # yanlış girildiğinde AuthenticationException
```

9.4. Access Control Testi

```
# Admin tüm katalogları görmeli
trino --user admin --execute "SHOW CATALOGS"

# Analyst sadece izinli katalogları görmeli
trino --user analyst1 --execute "SHOW CATALOGS"

# Column mask çalışıyor mu?
trino --user analyst1 --execute \
  "SELECT email FROM postgres.public.users LIMIT 5"
# Beklenen: email maskeli (r***@ethem.local gibi)

# Row filter çalışıyor mu?
```

```
trino --user analyst1 --execute \  
  "SELECT COUNT(*), country_code FROM postgres.public.users GROUP BY country_code"  
# Beklenen: sadece TR görünmeli
```

9.5. Resource Group Testi

```
# Airflow source ile query at → etl grubuna gitmeli  
trino --user ragnar \  
  --source "airflow-etl-daily" \  
  --execute "SELECT 1"  
  
# Ad-hoc source → adhoc grubuna gitmeli  
trino --user ragnar \  
  --source "dbeaver" \  
  --execute "SELECT 1"  
  
# system.runtime.queries'ten doğrula  
trino --user admin --execute "  
  SELECT query_id, user, source, resource_group_id, state  
  FROM system.runtime.queries  
  ORDER BY created DESC LIMIT 10  
"
```

9.6. Cluster Health (Cluster Kurulumları)

```
# Worker sayısını görüntüle  
curl -u admin:pass https://coordinator:8443/v1/cluster | jq .activeWorkers  
  
# Failed worker'lar  
curl -u admin:pass https://coordinator:8443/v1/node/failed | jq  
  
# Memory pool durumu  
curl -u admin:pass https://coordinator:8443/v1/memory | jq  
  
# JVM thread durumu  
curl -u admin:pass https://coordinator:8443/v1/thread | jq
```

10. Restart Gereksinimleri ve Değişiklik Yönetimi

Trino'da bazı dosyalar restart olmadan otomatik yenilenirken bazıları restart ister. Bu tablo değişiklik planlamasında yol göstericidir.

Dosya / Değişiklik	Restart	Notlar
node.properties	Evet	Sadece ilk açılışta okunur
jvm.config	Evet	JVM yeniden başlaması gerek
config.properties	Evet	Her değişiklik için
log.properties	Evet	Log seviyeleri restart'ta yüklenir
access-control.properties	Evet	Sadece bu dosya değişirse (rules.json değil)
rules.json	Hayır	refresh-period içinde otomatik
password-authenticator.properties	Evet	
password.db	Hayır	refresh-period içinde
group-provider.properties	Evet	
group.txt	Hayır	refresh-period içinde
resource-groups.properties	Evet	
resource-groups.json (file)	Hayır	~1 saniye içinde reload
Resource groups DB (db mode)	Hayır	Otomatik reload, HA için ideal
etc/catalog/*.properties (static)	Evet	
etc/catalog/*.properties (dynamic)	Hayır	CREATE/DROP CATALOG SQL ile
keystore.jks (cert rotation)	Evet	Cert yenileme için restart
TLS truststore	Evet	

10.1. Restart Komutları

```
# Docker
docker restart trino-secure-trino

# Docker Compose
docker-compose restart trino

# Systemd
sudo systemctl restart trino

# Manuel (binary install)
sudo -u trino /opt/trino/bin/launcher restart

# Graceful shutdown (worker'lar için)
curl -X PUT -u admin:pass \
  https://worker:8443/v1/info/state \
  -H "Content-Type: application/json" \
```

```
-d '"SHUTTING_DOWN"'  
# Bu, mevcut task'ları bitirmeyi bekler, sonra process ölür.
```

10.2. Değişiklik Yönetimi Önerileri

- Tüm konfigürasyon dosyalarını Git repo'da tut (secret'lar hariç).
- Prod'a apply etmeden önce dev/staging'de test et.
- Cluster'da rolling restart uygula: worker'ları teker teker graceful shutdown → restart.
- Coordinator restart'ı düşük trafik zamanında planla (aktif query'ler fail olur).
- Resource groups için DB manager kullan — konfigürasyon değişikliği restart'sız uygulanır.

11. Sonuç ve Öneriler

11.1. Genel Değerlendirme

Trino'nun güvenlik ve konfigürasyon mimarisi katmanlı bir yaklaşım sunar. Encryption, authentication ve authorization katmanları birbirinden bağımsız olarak yapılandırılabilir; bu esneklik, farklı ortam ve regülasyon gereksinimlerine uyum sağlar. Dosya-tabanlı konfigürasyon yaklaşımı basit ve versiyon kontrolüne uygun olsa da, üretim ortamlarında DB-tabanlı manager'lar (resource groups) ve harici authentication provider'lar (OAuth2, LDAP) tercih edilmelidir.

11.2. Üretim İçin Kritik Kontrol Listesi

- HTTPS zorunlu, HTTP tamamen kapalı olmalı
- internal-communication.shared-secret rastgele ve uzun (min 32 byte) üretilmeli
- Password auth yerine kurumsal SSO (OAuth2/OIDC) tercih edilmeli
- rules.json'da catch-all deny kuralı bulunmalı
- Katalog parolaları düz metin değil environment variable ile geçirilmeli
- Resource groups DB manager kullanılmalı (HA ve dinamik güncelleme için)
- Event listener ile audit trail aktif edilmeli
- JMX exporter + Prometheus + Grafana ile monitoring kurulmalı
- Cluster'da Trino Gateway ile coordinator HA sağlanmalı
- Konfigürasyon dosyaları Git repo'da versiyonlanmalı
- TLS sertifikaları için otomatik yenileme (cert-manager, ACME) kurulmalı

11.3. Sık Karşılaşılan Sorunlar

Sorun	Muhtemel Neden	Çözüm
Worker coordinator'a bağlanamıyor	shared-secret farklı veya discovery.uri yanlış	Tüm node'larda secret aynı mı, DNS/hostname resolution çalışıyor mu?
Static catalog store hatası	Coordinator dynamic, worker'lar static	Tüm node'larda catalog.management=dynamic ve aynı catalog.store dizini
TLS handshake fails	Hostname SAN'a eşleşmiyor	Keystore'u SAN'a doğru hostname ekleyerek yeniden oluştur
Password auth over HTTP hatası	HTTPS kapalı iken PASSWORD auth aktif	HTTPS aç veya allow-insecure-over-http (sadece test için)
Bcrypt cost hatası	Cost < 8	htpasswd -B -C 10 ile yeniden oluştur
Query resource group'a atanamıyor	Selector hiçbirine eşleşmiyor	En sona catch-all selector ekle
User "X-Trino-User must be sent"	CLI/JDBC'de user set edilmemiş	--user flag veya JDBC user property ekle

11.4. Sonraki Adımlar

Temel dosya-tabanlı konfigürasyon sağlam çalıştıktan sonra üretim ortamı için şu geçişler değerlendirilmelidir:

- Password authentication → OAuth2 (Keycloak, Okta, Azure AD)
- rules.json → Open Policy Agent (OPA) ile Rego policy'leri
- File-based resource groups → DB-backed manager
- Tek coordinator → Trino Gateway ile multi-coordinator HA
- Self-signed TLS → PKI ile yönetilen gerçek sertifikalar (cert-manager)
- Ad-hoc event listener → merkezi audit sistem (Ranger + Solr, SIEM entegrasyonu)

Bu rapor, Trino 481 sürümü referans alınarak hazırlanmıştır. Trino'nun aktif geliştirme sürecinde olduğu ve sürüm bazında farklılıklar bulunabileceği göz önünde bulundurulmalıdır. Güncel dokümantasyon için trino.io/docs/current adresine başvurulmalıdır.